Il branch-and-cut

Indice

- Perché gli algoritmi branch-and-cut?
- Algoritmi branch-and-cut
- Disuguaglianze valide globali e locali
- Cut-and-branch e branch-and-cut
  - Il pool di tagli
  - Pseudocodice ad alto livello
- Esempio svolto: cut-and-branch con tagli di Gomory
- Tagli locali: un esempio
- Aspetti pratici
  - Selezione dei tagli
  - Quali tagli
  - Upper bound e gap
- Errori tipici
- In sintesi

Obiettivi

Alla fine di questo capitolo sarai in grado di:

- spiegare come i piani di taglio e le operazioni di branching si completano a vicenda;
- distinguere le disuguaglianze valide globali e locali, e dire dove si può usare un taglio;
- descrivere gli algoritmi cut-and-branch e branch-and-cut, e il ruolo del pool di tagli;
- eseguire a mano un algoritmo cut-and-branch con tagli di Gomory su un piccolo problema PLI e confrontare la dimensione dell'albero con e senza tagli;
- generare un taglio locale in un nodo di branching e riottimizzare con l'algoritmo del simplesso duale.

Prerequisiti

Da questo modulo:

- I capitoli «Disuguaglianze valide» e «Disuguaglianze di cover»: le disuguaglianze valide e la separazione di una disuguaglianza violata di una famiglia
- Il capitolo «Algoritmo dei piani di taglio»: i tagli frazionari di Gomory e la riottimizzazione del problema LP con l'algoritmo del simplesso duale dopo l'inserimento di un taglio
- Il capitolo «Branch-and-bound basato sul rilassamento lineare»: la lista dei problemi LP, i criteri di potatura e le strategie di selezione della variabile e del nodo di branching

Perché gli algoritmi branch-and-cut?

Introduzione: Perché gli algoritmi branch-and-cut?

- Abbiamo visto due metodi esatti per i problemi PLI: l'algoritmo dei piani di taglio e l'algoritmo branch-and-bound
- Ognuno dei due ha un punto debole, e i punti deboli sono diversi
- Vediamo perché conviene combinarli, e incontriamo la sfida del capitolo

Due metodi esatti

- Consideriamo problemi PLI in **forma canonica di massimo con dati interi**, come nei capitoli sui piani di taglio e sul branch-and-bound

- I due metodi esatti visti finora attaccano il problema PLI da due lati diversi:
  - un **algoritmo dei piani di taglio** aggiunge disuguaglianze valide e avvicina il rilassamento lineare all'inviluppo convesso delle soluzioni intere; l'upper bound diminuisce
  - un **algoritmo branch-and-bound** suddivide la regione ammissibile in problemi LP più piccoli e li enumera implicitamente; lo spazio di ricerca viene partizionato

Due punti deboli

- Entrambi i metodi hanno un punto debole
- I tagli frazionari di Gomory perdono efficacia dopo le prime iterazioni (**tailing off**)
- Un algoritmo branch-and-bound che parte da un rilassamento lineare debole ha bisogno di un albero grande

Una sfida

Consideriamo il problema $P10$ del catalogo dei problemi MIP:
il suo albero di branch-and-bound ha 5 nodi
con le regole dell'esempio svolto.

Quanti nodi restano
se aggiungiamo **un taglio di Gomory** prima del branching?

- L'esempio svolto di questo capitolo risponde alla domanda

Tagli e branching insieme

- Un **algoritmo branch-and-cut** è un algoritmo branch-and-bound in cui i problemi LP dei nodi di branching sono rafforzati con piani di taglio prima del branching
- I tagli abbassano i bound dei nodi, così più nodi vengono potati per bound
- Il branching garantisce la terminazione quando i tagli non sono più efficaci
- Tutti i solver MIP moderni si basano su algoritmi branch-and-cut (si veda il capitolo «Solver MIP: cosa fanno»)

Riepilogo

- I piani di taglio **abbassano l'upper bound**, ma soffrono di tailing off
- Il branch-and-bound **partiziona lo spazio di ricerca**, ma con un rilassamento lineare debole l'albero è grande
- Un **algoritmo branch-and-cut** usa entrambi: tagli nei nodi, branching per terminare
- La sfida: di quanto un taglio di Gomory riduce l'albero di $P10$?

Algoritmi branch-and-cut

Introduzione: Algoritmi branch-and-cut

- Diamo la definizione formale di algoritmo branch-and-cut
- Vediamo come i piani di taglio e le operazioni di branching si completano a vicenda
- È lo schema su cui è costruito ogni solver MIP moderno

Definizione 1: algoritmo branch-and-cut

Dato un problema PLI in forma canonica di massimo con dati interi, un ***algoritmo branch-and-cut*** *è un algoritmo branch-and-bound in cui i problemi LP dei nodi di branching sono rafforzati con piani di taglio prima del branching.*

Due metodi che si completano a vicenda

- Un **algoritmo dei piani di taglio** avvicina il rilassamento lineare all'inviluppo convesso delle soluzioni intere, ma i tagli frazionari di Gomory perdono efficacia dopo le prime iterazioni (**tailing off**)

- Un **algoritmo branch-and-bound** suddivide la regione ammissibile in problemi LP più piccoli, ma partendo da un rilassamento lineare debole ha bisogno di un albero grande

Perché la combinazione funziona

I tagli abbassano i bound dei nodi,
così più nodi vengono **potati per bound**;

il branching garantisce la **terminazione**
quando i tagli non sono più efficaci.

- Tutti i solver MIP moderni si basano su algoritmi branch-and-cut (si veda il capitolo «Solver MIP: cosa fanno»)

Riepilogo

- Un algoritmo branch-and-cut è un **algoritmo branch-and-bound** i cui problemi LP dei nodi sono **rafforzati con tagli** prima del branching
- I tagli abbassano i bound dei nodi; il branching garantisce la terminazione
- È lo schema di tutti i solver MIP moderni

Disuguaglianze valide globali e locali

Introduzione: Disuguaglianze valide globali e locali

- In un nodo dell'albero, il problema LP contiene vincoli di branching
- Un taglio derivato con questi vincoli può essere valido solo sotto quel nodo
- Distinguiamo i tagli **globali** e **locali**, e vediamo dove si può usare ciascuno: un errore qui può eliminare la soluzione ottima

Le soluzioni intere di un nodo

- In un algoritmo branch-and-bound, il problema LP_k di un nodo di branching k è il rilassamento lineare con i vincoli di branching del cammino dalla radice al nodo k
- Indichiamo con F l'insieme delle soluzioni intere ammissibili del problema PLI
- Indichiamo con $F_k \subseteq F$ l'insieme di quelle che soddisfano i vincoli di branching del nodo k
- Nel nodo radice $F_1 = F$

Definizione 2: disuguaglianze valide globali e locali

Una disuguaglianza lineare $\alpha' \boldsymbol{x} \leq \beta$ *si chiama:*

1. ***disuguaglianza globalmente valida*** *(o* ***taglio globale****) se è soddisfatta da tutte le soluzioni in* F*;*
2. ***disuguaglianza localmente valida*** *per il nodo di branching* k *(o* ***taglio locale****) se è soddisfatta da tutte le soluzioni in* F_k*.*

Tagli globali e tagli locali

- Poiché $F_k \subseteq F$, ogni taglio globale è anche un taglio locale di ogni nodo
- Il viceversa è falso: un taglio locale può eliminare soluzioni intere ammissibili che appartengono ad altri rami dell'albero
- Un taglio locale del nodo k è valido per tutti i nodi del sottoalbero con radice in k, e non deve essere usato fuori da questo sottoalbero

Quando un taglio è globale?

Un taglio derivato solo dai vincoli originali
(e da altri tagli globali) è **globale**.

Un taglio la cui derivazione usa un vincolo di branching
è, in generale, solo **locale**.

Tagli di Gomory e disuguaglianze di cover

- Un taglio frazionario di Gomory generato nel nodo radice è **globale**: deriva dai soli vincoli originali
- Lo stesso vale per una disuguaglianza di una famiglia (come le disuguaglianze di cover di un vincolo a coefficienti positivi) separata a partire da essi
- Un taglio frazionario di Gomory generato da una riga che contiene la variabile di scarto di un vincolo di branching con coefficiente non nullo si tratta come **locale**

Riepilogo

Tagli globali e locali:

- Globale: soddisfatto da tutte le soluzioni in F; locale per il nodo k: da tutte le soluzioni in F_k
- Ogni taglio globale è locale per ogni nodo; il viceversa è falso

Dove si può usare un taglio:

- Un taglio locale del nodo k si usa solo nel sottoalbero con radice in k
- Un taglio la cui derivazione usa un vincolo di branching è, in generale, solo locale

Cut-and-branch e branch-and-cut

Introduzione: Cut-and-branch e branch-and-cut

- I tagli si possono generare solo nella radice, oppure anche negli altri nodi dell'albero
- Definiamo i due algoritmi, il **pool di tagli** che memorizza i tagli, e la sottoprocedura che risolve un nodo
- Lo pseudocodice è quello del branch-and-bound: cambia solo la soluzione di un nodo

Definizione 3: algoritmo cut-and-branch

*Un **algoritmo cut-and-branch** esegue un algoritmo dei piani di taglio nel nodo radice per un numero limitato di **round di tagli**, aggiunge al problema PLI i tagli trovati, e poi esegue un algoritmo branch-and-bound sul problema rafforzato. Negli altri nodi non si generano tagli.*

Tagli nella radice e negli altri nodi

- Tutti i tagli della radice sono globali: nel nodo radice non c'è alcun vincolo di branching
- In un **algoritmo branch-and-cut** i tagli si generano anche negli altri nodi (in tutti, o in alcuni, per esempio ogni dieci livelli dell'albero)
- I tagli possono allora essere globali o locali

Quando smettere di tagliare in un nodo

In ogni nodo la generazione dei tagli si ferma quando:

- la soluzione del problema LP_k è intera o il nodo può essere potato;
- non si trova alcun taglio violato;
- si raggiunge un numero massimo di round;
- la diminuzione di $z(\mathrm{LP}_k)$ negli ultimi round è piccola (tailing off).

Il pool di tagli (1/2)

- I tagli si memorizzano in una struttura dati chiamata **pool di tagli** (cut pool)
- Prima di chiamare le procedure (più costose) di generazione dei tagli, l'algoritmo controlla se un taglio del pool è violato da $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$
- I tagli globali si memorizzano in un **pool globale**, utilizzabile in ogni nodo
- I tagli locali si memorizzano con il loro nodo e sono ereditati solo dai suoi discendenti

Il pool di tagli (2/2)

- Ogni taglio aggiunge un vincolo ai problemi LP
- I tagli che restano inattivi per molte iterazioni si tolgono dal problema LP e si conservano solo nel pool

Pseudocodice ad alto livello

- L'algoritmo branch-and-cut ha lo pseudocodice dell'algoritmo branch-and-bound:
  - stesse operazioni di branching
  - stessi criteri di potatura
  - stessa lista L
- Cambia solo la **sottoprocedura di soluzione del nodo di branching**: la vediamo nelle prossime due slide

Sottoprocedura di soluzione del nodo di branching (1/2)

Input: Un problema LP_k associato a un nodo di branching, il pool di tagli e un numero massimo R di round di tagli.

Passo 1: **Operazioni di bounding**

Risolvere il problema LP_k.

(a) se il problema LP_k è inammissibile, allora `STOP`.

(b) se $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ è intera, allora, se $z(\mathrm{LP}_k) > \mathrm{LB}$, aggiornare il **lower bound** e la **soluzione incumbent**, togliere dalla lista L tutti i problemi LP con $\lfloor z(\mathrm{LP}) \rfloor \leq \mathrm{LB}$ e `STOP` (anche quando $z(\mathrm{LP}_k) \leq \mathrm{LB}$).

(c) se $\lfloor z(\mathrm{LP}_k) \rfloor \leq \mathrm{LB}$, allora `STOP`.

Sottoprocedura di soluzione del nodo di branching (2/2)

Passo 2: **Generazione dei tagli**

Se sono stati eseguiti R round di tagli nel nodo k, andare al Passo 3. Altrimenti:

(a) cercare nel pool di tagli i tagli violati da $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$; se non se ne trova nessuno, generare nuovi tagli violati e memorizzarli nel pool (i tagli globali nel pool globale, quelli locali con il nodo k);

(b) se non esiste alcun taglio violato, andare al Passo 3. Altrimenti aggiungere i tagli al problema LP_k, riottimizzarlo e andare al Passo 1.

Passo 3: **Inserimento nella lista**

Inserire il problema LP_k (con i suoi tagli) nella lista L e `STOP`.

Casi particolari e riottimizzazione

- Con $R = 0$ in ogni nodo tranne la radice otteniamo un **algoritmo cut-and-branch**
- Con $R = 0$ in ogni nodo otteniamo l'**algoritmo branch-and-bound**
- Lo pseudocodice non dice come si generano i tagli né come si riottimizza il problema LP
- Dopo l'inserimento di un taglio il dizionario è **duale ammissibile**, quindi l'**algoritmo del simplesso duale** parte dal dizionario ottimo del problema precedente, come dopo un vincolo di branching

Riepilogo

Due algoritmi:

- Cut-and-branch: round di tagli solo nella radice, poi branch-and-bound; tutti i tagli sono globali
- Branch-and-cut: tagli anche negli altri nodi, globali o locali

Il pool di tagli:

- Si controlla prima di generare nuovi tagli; pool globale e tagli locali memorizzati con il loro nodo

La sottoprocedura:

- Bounding, generazione dei tagli per al più R round, inserimento in L; riottimizzazione con il simplesso duale

Esempio svolto: cut-and-branch con tagli di Gomory

Introduzione: Esempio svolto

- Eseguiamo l'algoritmo branch-and-bound sul problema $P10$, prima senza tagli e poi dopo un taglio di Gomory nella radice
- Confrontiamo le dimensioni dei due alberi, e vediamo che cosa fa un secondo round di tagli
- Un secondo problema, $P2$, mostra che un taglio non riduce sempre l'albero

Il problema $P10$

- Usiamo il problema $P10$ del catalogo dei problemi MIP:

$$\begin{array}{rrcrcr} \max & 5\,x_1 & + & 2\,x_2 & & \\ \text{soggetto a} & -x_1 & + & 3\,x_2 & \leq & 12 \\ & x_1 & + & 2\,x_2 & \leq & 12 \\ & 3\,x_1 & - & 2\,x_2 & \leq & 8 \\ & x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0} \end{array}$$

- La sua unica soluzione ottima è $\tilde{\boldsymbol{x}}(\mathrm{ILP}) = (4, 4)$, con $z(\mathrm{ILP}) = 28$

Le regole delle esecuzioni

- I tagli sono i tagli frazionari di Gomory del capitolo sull'algoritmo dei piani di taglio, con la stessa regola: la riga con il termine noto frazionario più grande e, in caso di pareggio, l'indice minimo
- Poiché i dati sono interi, il bound di un nodo è $\lfloor z(\mathrm{LP}_k) \rfloor$
- Tutte le esecuzioni usano le stesse regole:
  - la regola dell'indice minimo come strategia di selezione della variabile di branching
  - la regola LIFO come strategia di selezione del nodo di branching
  - il nodo di branching verso il basso risolto per primo
  - $\mathrm{LB} = -\infty$ all'inizio

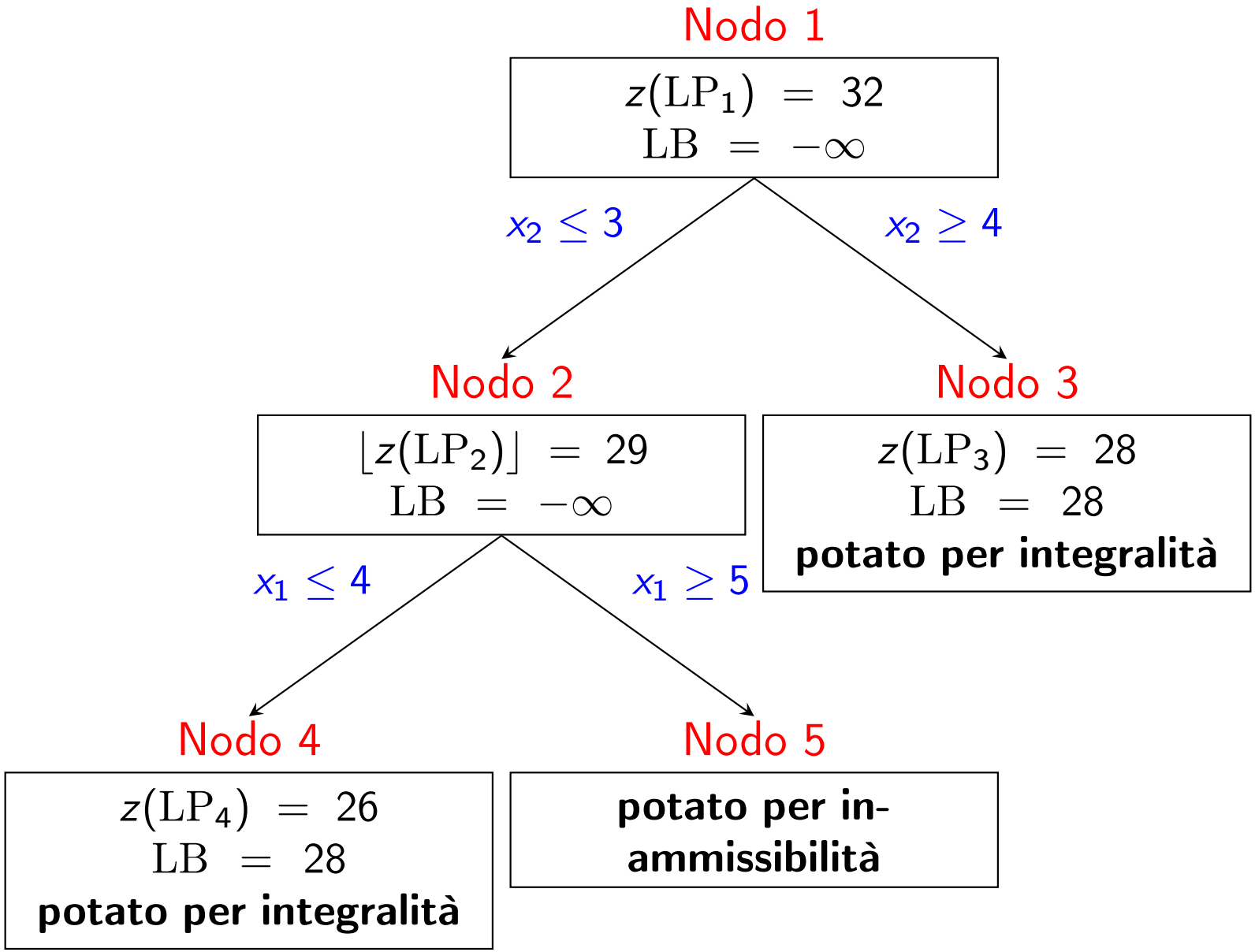
Esempio: branch-and-bound senza tagli (1/3)

- Il problema LP_1 è il rilassamento lineare di $P10$
- Una soluzione ottima è

$$\tilde{\boldsymbol{x}}(\mathrm{LP}_1) = (5, \tfrac{7}{2}), \qquad z(\mathrm{LP}_1) = 32, \qquad \mathrm{UB} = 32$$

- L'algoritmo branch-and-bound risolve 5 problemi LP: l'albero e la tabella dell'esecuzione sono nelle prossime due slide

Esempio: branch-and-bound senza tagli (2/3)



Esempio: branch-and-bound senza tagli (3/3)

Nodo	Padre	Branching	$\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$	$z(\mathrm{LP}_k)$	LB	Esito
1	–	–	$(5, \frac{7}{2})$	32	$-\infty$	branching su x_2
2	1	$x_2 \leq 3$	$(\frac{14}{3}, 3)$	$\frac{88}{3}$	$-\infty$	branching su x_1
3	1	$x_2 \geq 4$	$(4, 4)$	28	28	potato per integralità
4	2	$x_1 \leq 4$	$(4, 3)$	26	28	potato per integralità
5	2	$x_1 \geq 5$	–	–	28	potato per inammissibilità

Ora aggiungiamo un taglio di Gomory nella radice
prima del branching, ed eseguiamo di nuovo
lo stesso algoritmo branch-and-bound.

Esempio: cut-and-branch con un taglio di Gomory (1/5)

- **Radice: generazione dei tagli.** Le due righe del dizionario ottimo di LP_1 con termine noto frazionario, le righe di x_2 e di x_3, hanno lo stesso termine noto $\frac{1}{2}$
- Usiamo la riga di x_2, che ha l'indice minore
- Con le variabili di scarto $x_4 = 12 - x_1 - 2x_2$ e $x_5 = 8 - 3x_1 + 2x_2$ il taglio è

$$\tfrac{3}{8}\, x_4 + \tfrac{7}{8}\, x_5 \geq \tfrac{1}{2} \qquad \Longleftrightarrow \qquad 3\, x_1 - x_2 \leq 11.$$

- È un taglio globale, ed esclude $\tilde{\boldsymbol{x}}(\mathrm{LP}_1)$: $3 \cdot 5 - \frac{7}{2} = \frac{23}{2} > 11$

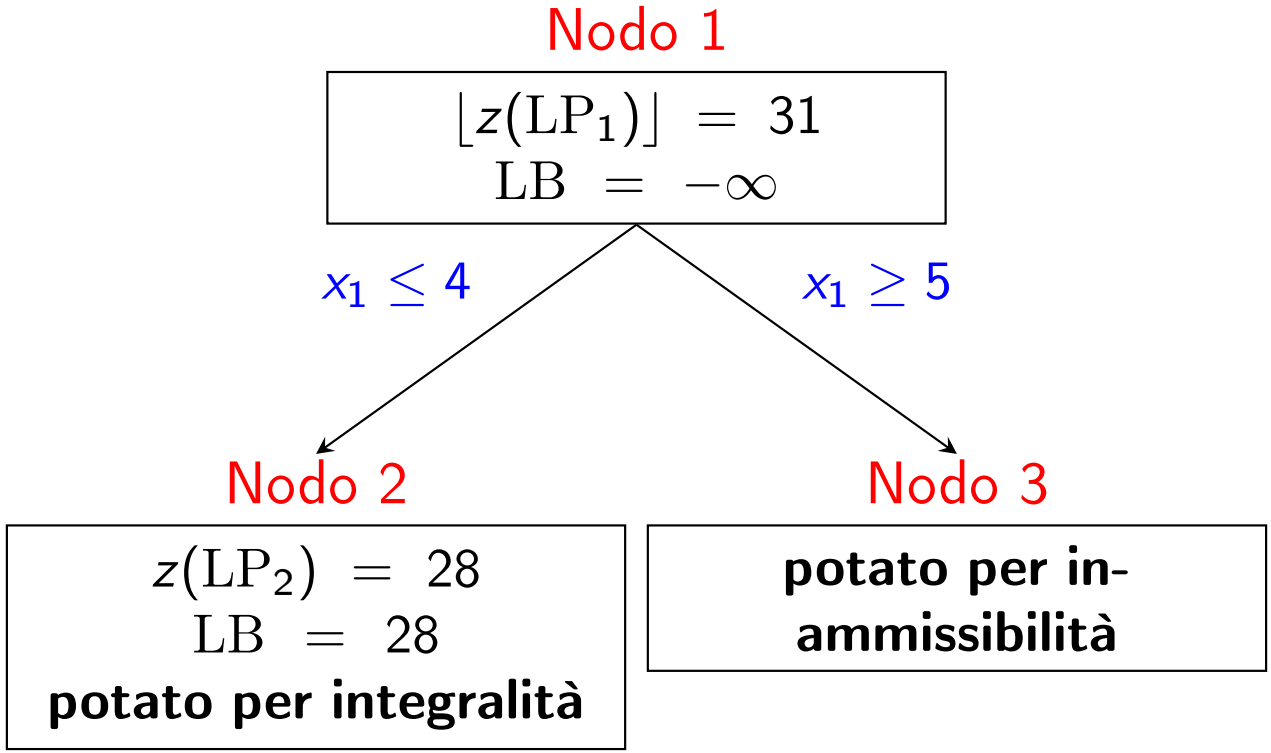
Esempio: cut-and-branch con un taglio di Gomory (2/5)

- **Radice: riottimizzazione.** Aggiungiamo il taglio al problema LP_1 e lo riottimizziamo con l'algoritmo del simplesso duale
- Una soluzione ottima è

$$\tilde{\boldsymbol{x}}(\mathrm{LP}_1) = (\tfrac{34}{7}, \tfrac{25}{7}), \qquad z(\mathrm{LP}_1) = \tfrac{220}{7}, \qquad \mathrm{UB} = 31.$$

- **Branch-and-bound.** L'algoritmo branch-and-bound parte ora da questo problema LP_1
- Ogni nodo eredita il taglio. Risolve 3 problemi LP invece di 5: albero e tabella nelle prossime slide

Esempio: cut-and-branch con un taglio di Gomory (3/5)



Esempio: cut-and-branch con un taglio di Gomory (4/5)

Nodo	Padre	Branching	$\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$	$z(\mathrm{LP}_k)$	LB	Esito
1	–	–	$(\frac{34}{7}, \frac{25}{7})$	$\frac{220}{7}$	$-\infty$	branching su x_1
2	1	$x_1 \leq 4$	$(4, 4)$	28	28	potato per integralità
3	1	$x_1 \geq 5$	–	–	28	potato per inammissibilità

Esempio: cut-and-branch con un taglio di Gomory (5/5)

- Un solo taglio di Gomory riduce il numero di problemi LP da 5 a 3
- Il taglio non trova la soluzione ottima: sposta la soluzione ottima del problema LP della radice
- Così l'algoritmo fa branching su x_1, e il nodo $x_1 \leq 4$ dà subito la soluzione ottima

Un secondo round di tagli

- Un secondo round di tagli, nello stesso algoritmo cut-and-branch, dà una soluzione intera nella radice
- La riga di x_1 ha il termine noto più grande, $\frac{6}{7}$
- Con la variabile di scarto $x_6 = 11 - 3x_1 + x_2$ del primo taglio il nuovo taglio è

$$\frac{1}{7}\, x_4 + \frac{2}{7}\, x_6 \geq \frac{6}{7} \qquad \Longleftrightarrow \qquad x_1 \leq 4.$$

La radice diventa intera

- La soluzione ottima del nuovo problema LP_1 è $(4, 4)$, con $z(\mathrm{LP}_1) = 28$
- Il nodo è potato per integralità, e l'algoritmo si ferma con $\mathrm{LB} = \mathrm{UB} = 28$ dopo aver risolto un solo problema LP
- La tabella della prossima slide confronta le tre esecuzioni su $P10$

Tre esecuzioni su $P10$

Tagli aggiunti nella radice	$z(\mathrm{LP}_1)$	UB nella radice	Problemi LP risolti
nessuno (branch-and-bound)	32	32	5
$3\,x_1 - x_2 \leq 11$	$\frac{220}{7}$	31	3
$3\,x_1 - x_2 \leq 11$ e $x_1 \leq 4$	28	28	1

Tagli più forti, alberi più piccoli?

- Su $P10$: più i tagli sono forti, più l'albero è piccolo, fino a ridursi al solo nodo radice
- Non è la regola
- Eseguiamo lo stesso algoritmo sul problema $P2$ del catalogo, per un numero crescente di round di tagli di Gomory

Il problema $P2$

- Il problema $P2$ del catalogo dei problemi MIP:

$$
\begin{array}{rrcrcr}
\max & x_1 & + & 3\,x_2 & & \\
\text{soggetto a} & -4\,x_1 & + & 10\,x_2 & \leq & 35 \\
& 3\,x_1 & + & x_2 & \leq & 12 \\
& x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0}
\end{array}
$$

- Il suo valore ottimo è $z(\mathrm{ILP}) = 14$

Round di tagli di Gomory su $P2$

Round	Ultimo taglio aggiunto	$z(\mathrm{LP}_1)$	UB nella radice	Problemi LP risolti
0	–	16	16	4
1	$-3\,x_1 + 10\,x_2 \leq 37$	$\frac{524}{33}$	15	4
2	$x_1 + 3\,x_2 \leq 15$	15	15	4
3	$-x_1 + 5\,x_2 \leq 19$	15	15	4
4	$x_2 \leq 4$	$\frac{44}{3}$	14	2
5	$x_1 + x_2 \leq 6$	14	14	1

Tailing off su $P2$

- I primi tre tagli abbassano $z(\mathrm{LP}_1)$ da 16 a 15, ma l'albero ha sempre 4 nodi
- Servono 5 round per arrivare al solo nodo radice
- L'upper bound migliora lentamente, come nel **tailing off** dell'algoritmo dei piani di taglio
- Un algoritmo cut-and-branch deve decidere quando smettere di tagliare e cominciare il branching: il costo di ogni round si paga in ogni nodo dell'albero

Riepilogo

Su $P10$:

- Branch-and-bound senza tagli: 5 problemi LP
- Un taglio di Gomory nella radice, $3x_1 - x_2 \leq 11$: 3 problemi LP
- Un secondo taglio, $x_1 \leq 4$: la radice è intera, 1 problema LP

Su $P2$:

- I primi tre tagli lasciano l'albero a 4 nodi; con 5 round si arriva al solo nodo radice
- Quando smettere di tagliare e cominciare il branching è una decisione dell'algoritmo

Tagli locali: un esempio

Introduzione: Tagli locali: un esempio

- Torniamo all'esempio dell'auto, in un nodo sotto la radice
- Deriviamo un taglio locale da un vincolo di branching, lo aggiungiamo e riottimizziamo con l'algoritmo del simplesso duale
- Poi vediamo un taglio di Gomory nello stesso nodo che l'algoritmo deve trattare come locale

L'esempio dell'auto

- Usiamo l'esempio dell'auto (x_1 auto sportive e x_2 auto familiari):

$$\begin{array}{rrcrcr} \max & 3\,x_1 & + & 2\,x_2 & & \\ \text{soggetto a} & 10\,x_1 & + & 4\,x_2 & \leq & 45 \\ & 4\,x_1 & + & 14\,x_2 & \leq & 49 \\ & x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0} \end{array}$$

- Nell'algoritmo branch-and-bound senza tagli, il nodo 3 è il figlio verso l'alto della radice, con il vincolo di branching $x_1 \geq 4$ e la variabile di scarto $x_5 = x_1 - 4$

Il dizionario ottimo di LP_3

- Il dizionario ottimo del problema LP_3, con $\tilde{\boldsymbol{x}}(\mathrm{LP}_3) = (4, \frac{5}{4})$ e $z(\mathrm{LP}_3) = \frac{29}{2}$, è:

$$
\begin{cases}
x_1 = 4 + x_5 \\
x_2 = \frac{5}{4} - \frac{1}{4} x_3 - \frac{5}{2} x_5 \\
x_4 = \frac{31}{2} + \frac{7}{2} x_3 + 31 x_5 \\
z = \frac{29}{2} - \frac{1}{2} x_3 - 2 x_5
\end{cases}
$$

Esempio: un taglio locale nel nodo 3 (1/7)

- **Un taglio locale semplice.** Da $10\,x_1 + 4\,x_2 \leq 45$ e dal vincolo di branching $x_1 \geq 4$ abbiamo

$$4\,x_2 \leq 45 - 10\,x_1 \leq 45 - 10 \cdot 4 = 5 \quad \Longrightarrow \quad x_2 \leq \tfrac{5}{4} \quad \Longrightarrow \quad x_2 \leq 1.$$

Esempio: un taglio locale nel nodo 3 (2/7)

- La disuguaglianza $x_2 \leq 1$ è valida per tutte le soluzioni in F_3, ma **non è globalmente valida**
- La soluzione intera ammissibile $(3, 2)$, di valore 13, la viola
- È un taglio locale del nodo 3
- La aggiungiamo al problema LP_3, con la variabile di scarto $x_6 = 1 - x_2$, e riottimizziamo

Esempio: un taglio locale nel nodo 3 (3/7)

- Il taglio si inserisce con la variabile di scarto $x_6 \geq 0$. La nuova riga è

$$x_6 = -\frac{1}{4} + \frac{1}{4}\, x_3 + \frac{5}{2}\, x_5.$$

- Il dizionario della prossima slide è **duale ammissibile** ma non primale ammissibile, perché

$$\bar{b}_6 = -\frac{1}{4} < 0$$

Esempio: un taglio locale nel nodo 3 (4/7)

$$
\begin{cases}
x_1 = 4 + x_5 \\
x_2 = \frac{5}{4} - \frac{1}{4} x_3 - \frac{5}{2} x_5 \\
x_4 = \frac{31}{2} + \frac{7}{2} x_3 + 31 x_5 \\
x_6 = -\frac{1}{4} + \frac{1}{4} x_3 + \frac{5}{2} x_5 \\
z = \frac{29}{2} - \frac{1}{2} x_3 - 2 x_5
\end{cases}
$$

Esempio: un taglio locale nel nodo 3 (5/7)

- **Operazione di pivot duale.** La riga pivot è quella di x_6, l'unica variabile in base con valore negativo
- I rapporti delle variabili fuori base con $\bar{a}_{6j} < 0$ sono

$$\frac{\bar{c}_3}{\bar{a}_{6,3}} = \frac{-\frac{1}{2}}{-\frac{1}{4}} = 2, \qquad \frac{\bar{c}_5}{\bar{a}_{6,5}} = \frac{-2}{-\frac{5}{2}} = \frac{4}{5}.$$

- La colonna pivot è quella di x_5, con $s_m = \frac{4}{5}$, e il valore obiettivo diventa

$$\bar{z} + s_m \, \bar{b}_6 = \frac{29}{2} + \frac{4}{5} \cdot \left(-\frac{1}{4}\right) = \frac{143}{10}$$

Esempio: un taglio locale nel nodo 3 (6/7)

- Il nuovo dizionario è primale e duale ammissibile, quindi ottimo:

$$\begin{cases} x_1 = \frac{41}{10} - \frac{1}{10}x_3 + \frac{2}{5}x_6 \\ x_2 = 1 - x_6 \\ x_4 = \frac{93}{5} + \frac{2}{5}x_3 + \frac{62}{5}x_6 \\ x_5 = \frac{1}{10} - \frac{1}{10}x_3 + \frac{2}{5}x_6 \\ z = \frac{143}{10} - \frac{3}{10}x_3 - \frac{4}{5}x_6 \end{cases}$$

Esempio: un taglio locale nel nodo 3 (7/7)

- La soluzione $\tilde{\boldsymbol{x}}(\mathrm{LP}_3) = (\frac{41}{10}, 1)$, di valore $\frac{143}{10}$, è la stessa che dà il figlio verso il basso del nodo 3
- Qui il taglio locale fa da vincolo di branching, senza creare un nuovo nodo

Il taglio $x_2 \leq 1$ è stato derivato a mano.

E un taglio frazionario di Gomory
generato dal dizionario del nodo 3?

Esempio: un taglio frazionario di Gomory nel nodo 3 (1/2)

- La riga di x_2 del dizionario del nodo 3 dà il taglio frazionario di Gomory

$$\frac{1}{4}x_3 + \frac{1}{2}x_5 \geq \frac{1}{4}$$

- Contiene la variabile di scarto x_5 del vincolo di branching $x_1 \geq 4$, quindi la sua derivazione usa questo vincolo e l'algoritmo lo tratta come un **taglio locale**

Esempio: un taglio frazionario di Gomory nel nodo 3 (2/2)

- Nello spazio delle variabili originali ($x_3 = 45 - 10\,x_1 - 4\,x_2$, $x_5 = x_1 - 4$, moltiplicando per 4) si scrive

$$2\,x_1 + x_2 \leq 9$$

- Enumerando le 16 soluzioni intere ammissibili si vede che questa disuguaglianza è valida per tutte

- L'algoritmo non può fare questo controllo in un nodo, quindi tiene il taglio come locale

Riepilogo

Un taglio locale nel nodo 3:

- Da $10x_1 + 4x_2 \leq 45$ e $x_1 \geq 4$: il taglio $x_2 \leq 1$, valido in F_3 ma violato da $(3, 2)$
- Un'operazione di pivot duale dà $(\frac{41}{10}, 1)$, di valore $\frac{143}{10}$: il taglio fa da vincolo di branching

Un taglio di Gomory nel nodo 3:

- Contiene la variabile di scarto x_5 di $x_1 \geq 4$: si tratta come locale
- In realtà è globale ($2x_1 + x_2 \leq 9$), ma l'algoritmo non può saperlo

Aspetti pratici

Introduzione: Aspetti pratici

- Un vero algoritmo branch-and-cut deve scegliere quali tagli aggiungere, fra molti
- Vediamo come si misurano i tagli (**efficacia**), quali famiglie di tagli si usano, e come i tagli cambiano l'upper bound e il gap
- Queste scelte decidono le prestazioni di un solver MIP

Selezione dei tagli

- A ogni round possono essere disponibili molti tagli violati
- Aggiungerli tutti rende i problemi LP grandi e lenti da riottimizzare
- Un algoritmo branch-and-cut ne seleziona un sottoinsieme

L'efficacia di un taglio

- L'**efficacia** di un taglio $\boldsymbol{\alpha}'\boldsymbol{x} \leq \beta$ nella soluzione frazionaria $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ è la distanza euclidea di $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ dall'iperpiano $\boldsymbol{\alpha}'\boldsymbol{x} = \beta$:

$$\text{efficacia} = \frac{\boldsymbol{\alpha}'\,\tilde{\boldsymbol{x}}(\mathrm{LP}_k) - \beta}{\|\boldsymbol{\alpha}\|_2}.$$

- A differenza della violazione $\boldsymbol{\alpha}'\tilde{\boldsymbol{x}}(\mathrm{LP}_k) - \beta$, l'efficacia non cambia se il taglio si moltiplica per una costante positiva

- Le regole di selezione combinano l'efficacia con una misura del **parallelismo** dei tagli

Efficacia e violazione su $P10$

- In $\tilde{\boldsymbol{x}}(\mathrm{LP}_1) = (5, \frac{7}{2})$ di $P10$:
- il taglio $3x_1 - x_2 \leq 11$ ha violazione $\frac{1}{2}$ ed efficacia

$$\frac{1}{2\sqrt{10}} \approx 0.158$$

- lo stesso taglio moltiplicato per 2, $6x_1 - 2x_2 \leq 22$, ha violazione 1 e la stessa efficacia

$$\frac{1}{\sqrt{40}} \approx 0.158$$

Quali tagli

- Si può usare qualunque famiglia di disuguaglianze valide con una procedura di separazione:
  - i tagli frazionari di Gomory (per problemi PLI con dati interi)
  - le disuguaglianze di cover per i vincoli $\sum_j a_j x_j \leq b$ a coefficienti positivi di un problema MIP con variabili binarie
  - famiglie che sfruttano la struttura di un modello specifico

- I tagli frazionari di Gomory vengono dalle righe del dizionario ottimo, quindi sono sempre disponibili

- In aritmetica floating point i loro coefficienti risentono degli errori di arrotondamento, e i solver scartano i tagli con coefficienti molto grandi o molto piccoli

Upper bound e gap

- Come nell'algoritmo branch-and-bound, se la lista L non è vuota abbiamo

$$z(\mathrm{ILP}) \;\leq\; \mathrm{UB} \;=\; \max_{k \in L} \{\, \mathrm{UB}(\mathrm{LP}_k) \,\},$$

dove ora $z(\mathrm{LP}_k)$ è il valore ottimo del problema LP_k dopo i tagli

- I tagli abbassano i valori $z(\mathrm{LP}_k)$ e, di conseguenza, l'upper bound UB e il gap di ottimalità

$$\frac{\mathrm{UB} - \mathrm{LB}}{|\mathrm{LB}|}$$

Riepilogo

Selezione dei tagli:

- Si aggiunge solo un sottoinsieme dei tagli violati
- Efficacia: distanza di $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ dall'iperpiano del taglio, invariante per moltiplicazione; combinata con il parallelismo

Quali tagli:

- Tagli frazionari di Gomory, disuguaglianze di cover, famiglie di un modello specifico; i tagli numericamente pericolosi si scartano

Bound:

- I tagli abbassano $z(\mathrm{LP}_k)$, l'upper bound UB e il gap

Errori tipici

Introduzione: Errori tipici

- Elenchiamo gli errori che si fanno più spesso con gli algoritmi branch-and-cut
- Riguardano la validità dei tagli, quanti tagli aggiungere, come confrontarli e che cosa cambiano i tagli
- Conoscerli aiuta a leggere e risolvere correttamente gli esercizi

Errori tipici

Osservazione 1: Errori tipici (1/3)

- ***Usare un taglio locale fuori dal suo sottoalbero.*** *Un taglio derivato con un vincolo di branching del nodo k è valido solo nel sottoalbero del nodo k: in un altro ramo può eliminare una soluzione ottima.*
- ***Trattare un taglio come globale senza una dimostrazione.*** *Un taglio frazionario di Gomory generato in un nodo può essere globale per caso (il taglio di Gomory nel nodo 3 dell'esempio dell'auto); l'algoritmo lo tiene locale, a meno che la sua derivazione non usi vincoli di branching.*

Errori tipici

Osservazione: Errori tipici (2/3)

- ***Aggiungere tutti i tagli violati.*** *Troppi tagli rallentano i problemi LP; meglio pochi tagli efficaci per round, togliendo quelli inattivi.*
- ***Credere che ogni taglio riduca l'albero.*** *Su P2 i primi tre tagli di Gomory lasciano l'albero a 4 nodi (tabella dell'esempio svolto); conta la qualità dei tagli.*

Errori tipici

Osservazione: Errori tipici (3/3)

- ***Scegliere come più forte il taglio di efficacia massima.*** *L'efficacia si misura solo nel punto corrente: un taglio dominato può avere in quel punto un'efficacia maggiore.*
- ***Dimenticare che i tagli abbassano il bound, non il valore ottimo.*** *Il valore ottimo* $z(\mathrm{ILP})$ *non cambia mai: cambiano solo* $z(\mathrm{LP}_k)$ *e l'upper bound.*

Riepilogo

Errori tipici:

- Un taglio locale si usa solo nel suo sottoalbero, e un taglio è globale solo con una dimostrazione
- Pochi tagli efficaci sono meglio di tutti i tagli violati
- Non ogni taglio riduce l'albero, e l'efficacia si misura solo nel punto corrente
- I tagli cambiano il bound, mai $z(\mathrm{ILP})$

In sintesi

Introduzione: In sintesi

- Raccogliamo le idee chiave del capitolo
- Chiudiamo con il cruscotto di $P10$: i bound trovati in questo capitolo

Riepilogo (1/3)

- Un algoritmo branch-and-cut rafforza con tagli il problema LP di ogni nodo prima del branching
- Un taglio globale è valido per tutte le soluzioni intere; un taglio locale solo per quelle di un sottoalbero, e si memorizza con il suo nodo
- Un algoritmo cut-and-branch genera tagli solo nella radice; i tagli sono quindi globali

Riepilogo (2/3)

- Dopo un taglio, il problema LP si riottimizza con l'algoritmo del simplesso duale a partire dal dizionario precedente
- I tagli riducono la dimensione dell'albero, ma solo se sono efficaci: su $P10$ un taglio di Gomory riduce l'albero da 5 a 3 problemi LP, e due tagli a 1
- Come i solver MIP combinano queste idee è l'argomento del capitolo «Solver MIP»

Riepilogo (3/3)

Cruscotto di $P10$. I lower e upper bound su $z(\mathrm{ILP}) = 28$ trovati in questo capitolo:

Fase	Lower bound	Upper bound	Da
Rilassamento lineare di $P10$	—	32	$z(\mathrm{LP}) = 32$, $\tilde{\boldsymbol{x}}(\mathrm{LP}) = (5, \frac{7}{2})$
Branch-and-bound	28	28	5 problemi LP risolti
Dopo il primo taglio di Gomory	—	31	$z(\mathrm{LP}_1) = \frac{220}{7}$
Cut-and-branch con un solo taglio	28	28	3 problemi LP risolti
Dopo il secondo taglio di Gomory	28	28	$\tilde{\boldsymbol{x}}(\mathrm{LP}_1) = (4, 4)$ è intera, 1 problema LP risolto